Ministerul Educației și Cercetării Al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 4:
Dynamic Programming: Dijkstra and Floyd-Warshall Algorithms

Elaborated:
st. gr. FAF-243 Poiata Calin

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Table of Contents

ALGORITHM ANALYSIS **3**
Objective 3
Tasks 3
Theoretical Notes 3
Introduction 3
Comparison Metric 4
Input Data 4
IMPLEMENTATION **5**
Dijkstra Algorithm 5
Floyd-Warshall Algorithm 6
RESULTS **7**
Execution Time Table 7
Standard Deviation Table 7
Performance Ratios 8
Graphical Analysis 8
CONCLUSION **10**
ANNEX **11**

ALGORITHM ANALYSIS

Objective

Study dynamic programming techniques through the implementation and empirical analysis of Dijkstra and Floyd-Warshall shortest-path algorithms on sparse and dense graphs.

Tasks

1. Study the dynamic programming method of algorithm design;
2. Implement Dijkstra and Floyd-Warshall shortest-path algorithms in Python;
3. Perform empirical analysis for sparse and dense graphs;
4. Increase the number of nodes and analyze the impact on execution time;
5. Present experimental results in tables and graphical form;
6. Formulate conclusions based on the obtained data.

Theoretical Notes

Dynamic programming solves complex problems by decomposing them into overlapping subproblems, storing partial results, and combining them to build an optimal global answer.

For shortest paths:

- **Dijkstra** uses optimal substructure from a source node. Once the shortest distance to a node is finalized, it is not improved later (for non-negative weights).
- **Floyd-Warshall** is a classic all-pairs dynamic programming algorithm with state

  `dist[i][j][k]`

  : the shortest path from i to j using only intermediate nodes from $\{1, 2, ..., k\}$.

Standard methodology:

1. Define the analysis objective (execution time of shortest-path algorithms).
2. Select a metric (average wall-clock time in milliseconds).
3. Define input properties (graph density and number of nodes).
4. Implement the algorithms.
5. Generate sparse and dense weighted directed graphs.
6. Execute experiments for increasing graph sizes and multiple trials.
7. Analyse, tabulate, and visualise the measurements.

Introduction

Dijkstra and Floyd-Warshall solve related shortest-path problems but target different use cases. Dijkstra computes shortest paths from one source to all nodes and is highly efficient on sparse graphs when implemented with a priority queue. Floyd-Warshall computes shortest paths between all pairs of nodes and has a cubic time complexity, making it significantly more expensive as the graph grows.

This laboratory examines:

- the impact of graph density (sparse vs dense),
- the impact of increasing number of nodes,
- and the practical gap between asymptotic complexity and measured runtime.

Comparison Metric

The primary metric is **average wall-clock execution time** (seconds), measured with `time.perf_counter()`. Standard deviation is also recorded to estimate run-to-run variation.

Additional derived metrics are included:

- Dijkstra dense/sparse runtime ratio,
- Floyd-Warshall dense/sparse runtime ratio,
- Dijkstra/Floyd-Warshall ratio on sparse and dense graphs.

Input Data

The benchmark script tests the following node sizes:

- $N \in \{10, 20, 50, 100, 150, 200\}$

For each size, two directed weighted graphs are generated:

1. **Sparse graph**: approximately $2N$ edges, represented in an adjacency matrix with ∞ for absent edges;
2. **Dense graph**: each directed edge $(i, j), i \neq j$ is added with probability $p = 0.5$;
3. Edge weights are random integers in $[1, 100]$;
4. Trials per size: 3 trials for $N \leq 100$, 2 trials for $N > 100$.

IMPLEMENTATION

Both algorithms are implemented in Python and tested by an automated benchmarking harness. Graphs are generated deterministically per trial using seed control (

```
42 + trial
```

) to keep experiments reproducible.

Dijkstra Algorithm

Algorithm Description:

Dijkstra solves the **single-source shortest path** problem for non-negative edge weights. The implementation uses a binary min-heap (

```
heapq
```

) and repeatedly relaxes outgoing edges.

Complexity with heap: $O((V + E) \log V)$, with $O(V)$ additional memory.

Pseudocode:

```
Dijkstra(graph, source):
    dist <- [∞] * n
    dist[source] <- 0
    visited <- [False] * n
    pq <- [(0, source)]

    while pq not empty:
        (d, u) <- heappop(pq)
        if visited[u]: continue
        visited[u] <- True

        for each vertex v:
            if edge(u, v) exists and not visited[v]:
                if d + w(u, v) < dist[v]:
                    dist[v] <- d + w(u, v)
                    heappush(pq, (dist[v], v))

    return dist
```

Implementation:

```python
def solve(graph, source=0):
    n = len(graph)
    distances = [float('inf')] * n
    distances[source] = 0
    visited = [False] * n
    pq = [(0, source)]

    while pq:
        current_dist, u = heapq.heappop(pq)
        if visited[u]:
```

```
            continue
        visited[u] = True

        for v in range(n):
            if graph[u][v] != float('inf') and not visited[v]:
                new_dist = current_dist + graph[u][v]
                if new_dist < distances[v]:
                    distances[v] = new_dist
                    heapq.heappush(pq, (new_dist, v))

    return distances
```

Floyd-Warshall Algorithm

Algorithm Description:

Floyd-Warshall solves the **all-pairs shortest path** problem by dynamic programming on the set of allowed intermediate vertices.

Transition:

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

Complexity: $O(V^3)$ time and $O(V^2)$ memory.

Pseudocode:

```
FloydWarshall(graph):
    dist <- copy(graph)
    for k in 0..n-1:
        for i in 0..n-1:
            for j in 0..n-1:
                if dist[i][k] and dist[k][j] are finite:
                    dist[i][j] <- min(dist[i][j], dist[i][k] + dist[k][j])
    return dist
```

Implementation:

```
def solve(graph):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != float('inf') and dist[k][j] != float('inf'):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist
```

RESULTS

The program was executed for all configured node sizes and both graph densities. Execution below reflects direct run of

```
program.py
```

in the project virtual environment.

Execution Time Table

Nodes	Dijkstra (Sparse)	Dijkstra (Dense)	Floyd-W (Sparse)	Floyd-W (Dense)
10	0.000016	0.000020	0.000199	0.000304
20	0.000045	0.000070	0.001431	0.002489
50	0.000215	0.000390	0.016707	0.038875
100	0.000780	0.001438	0.123694	0.313196
150	0.000860	0.003202	0.415566	1.049849
200	0.003076	0.005556	0.978955	2.499071

Figure 1 Average execution time (seconds) for Dijkstra and Floyd-Warshall

Measured values from the run are:

1. At $N = 200$, Dijkstra: 0.003074 s (sparse), 0.005548 s (dense);
2. At $N = 200$, Floyd-Warshall: 0.978077 s (sparse), 2.514712 s (dense);
3. Floyd-Warshall is orders of magnitude slower than Dijkstra for all tested sizes;
4. Dense graphs are slower than sparse graphs for both algorithms at every tested size.

Standard Deviation Table

Nodes	Dijkstra (Sparse)	Dijkstra (Dense)	Floyd-W (Sparse)	Floyd-W (Dense)
10	0.000003	0.000001	0.000010	0.000013
20	0.000005	0.000001	0.000139	0.000093
50	0.000028	0.000016	0.000385	0.000227
100	0.000005	0.000048	0.006644	0.000746
150	0.000825	0.000029	0.019323	0.000252
200	0.000169	0.000066	0.035360	0.000968

Figure 2 Runtime standard deviations (seconds)

The standard deviations remain relatively small compared to means, so the trend is stable and not caused by random fluctuations.

Performance Ratios

Nodes	Dijkstra (Sparse vs Dense)	Floyd-W (Sparse vs Dense)	Dijkstra vs Floyd-W (Sparse)	Dijkstra vs Floyd-W (Dense)
10	1.19x	1.53x	0.08x	0.06x
20	1.56x	1.74x	0.03x	0.03x
50	1.81x	2.33x	0.01x	0.01x
100	1.84x	2.53x	0.01x	0.00x
150	3.72x	2.53x	0.00x	0.00x
200	1.81x	2.55x	0.00x	0.00x

Figure 3 Derived performance ratios (dense/sparse and Dijkstra/Floyd-Warshall)

Important ratio-based observations:

1. Dijkstra dense/sparse ratio is between approximately 1.19x and 3.72x;
2. Floyd-Warshall dense/sparse ratio is between approximately 1.52x and 2.57x;
3. Dijkstra/Floyd-Warshall ratio is near 0.00x-0.08x, confirming a large performance gap.

Graphical Analysis

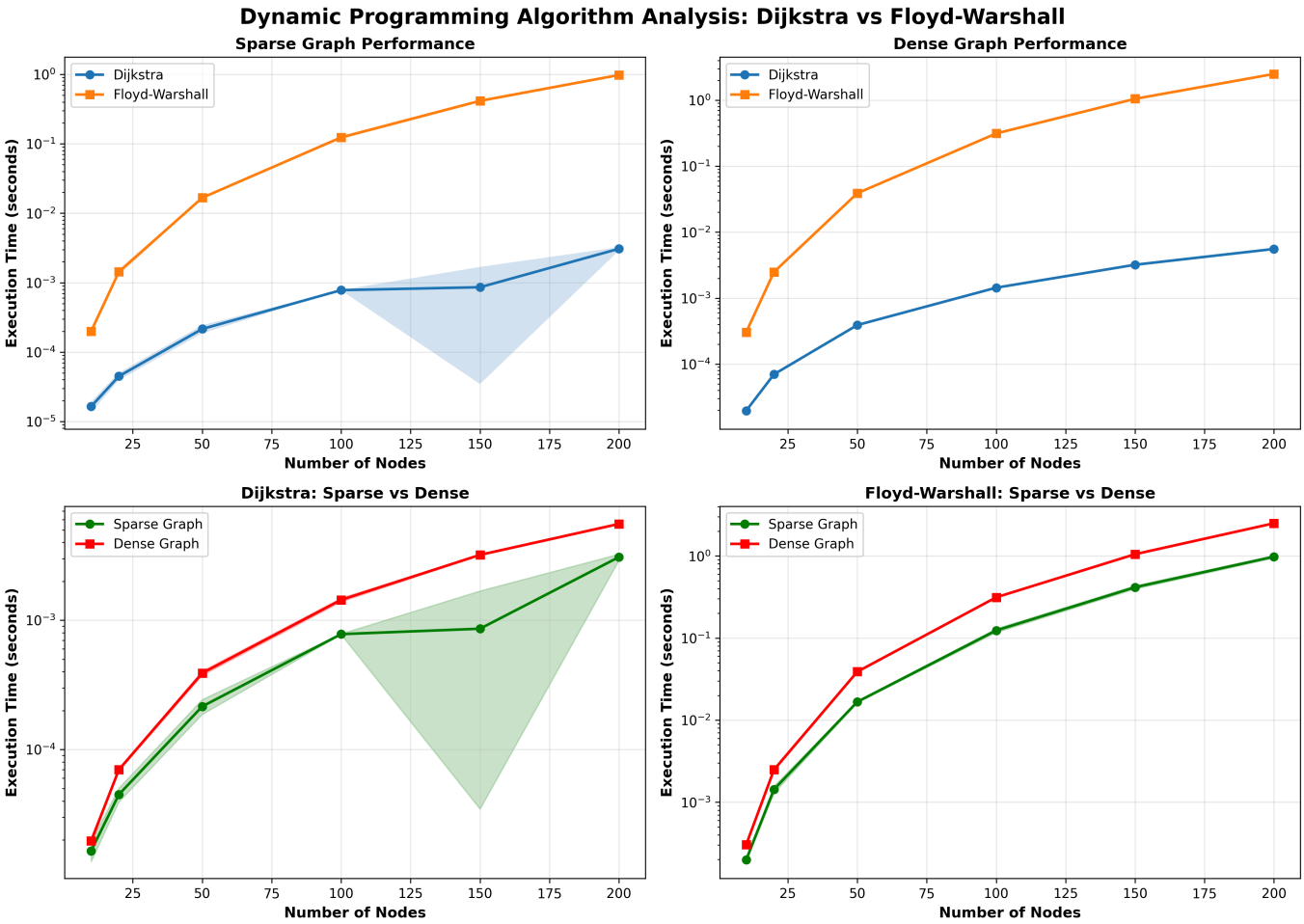

Figure 4 Runtime scaling with node count for sparse and dense graphs

From the plots:

1. Dijkstra curves increase moderately with N and stay in the millisecond range up to 200 nodes;
2. Floyd-Warshall curves rise steeply (roughly cubic trend), crossing near and above 1 second;
3. Dense graph curves are consistently above sparse graph curves;

4. Increasing nodes has a much stronger impact on Floyd-Warshall than on Dijkstra.

CONCLUSION

The laboratory objectives were achieved. Dynamic programming principles were studied and applied, both Dijkstra and Floyd-Warshall algorithms were implemented in Python, and an empirical analysis was carried out for sparse and dense graphs with increasing numbers of nodes. The obtained tables and plots allowed a direct comparison between theoretical expectations and practical behavior.

The results show that **Dijkstra is substantially faster** than Floyd-Warshall in all tested configurations. For $N = 200$, Dijkstra remains in the millisecond range, while Floyd-Warshall requires approximately 1 to 2.5 seconds depending on graph density. The experiments also confirm that increasing graph density increases runtime for both algorithms across all tested values of N.

Scalability differs significantly between the two methods. Dijkstra grows much more slowly with input size, while Floyd-Warshall follows the expected $O(V^3)$ trend and becomes expensive as the number of nodes grows. Therefore, for practical shortest-path tasks on larger graphs, especially sparse ones, Dijkstra is the recommended choice. Floyd-Warshall remains appropriate when all-pairs shortest paths are required and the graph size is limited.

Overall, the experimental results are consistent with theoretical complexity analysis and confirm the importance of selecting the algorithm based on both task type (single-source vs all-pairs) and graph characteristics (size and density).

ANNEX

Github Repository